

Balanced Binary Search Trees

2026-09-19

A binary search tree keeps its keys in sorted order, so lookup, insertion and deletion can skip half of the remaining tree at every step. That promise only holds while the tree stays balanced. A balanced binary search tree maintains logarithmic height by performing rotations after insertions and deletions. The most common variants are AVL trees and red-black trees, and both guarantee the bound below for every sequence of operations.

ask about AVL

$$h = O(\log n)$$

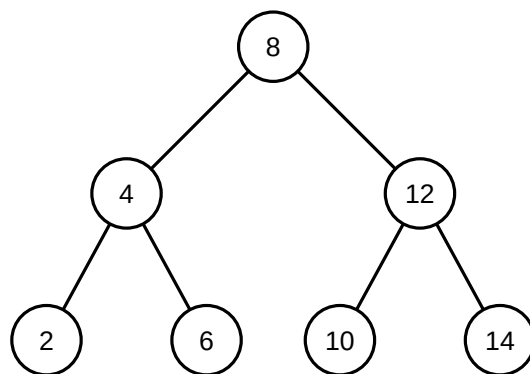


Fig 3. A balanced binary search tree of height 2

Rotations

A rotation changes the shape of a subtree without changing the in-order sequence of its keys. A single left rotation lifts the right child into the parent's place and makes the old parent its left child.

$$x^2 + y^2 = r^2$$

Rotations are constant-time operations, so the cost of rebalancing is dominated by walking back up the path from the modified leaf to the root.

Red-black trees

A red-black tree colours every node and enforces that no red node has a red child. The longest path is therefore at most twice the shortest, which is enough to keep the height logarithmic.

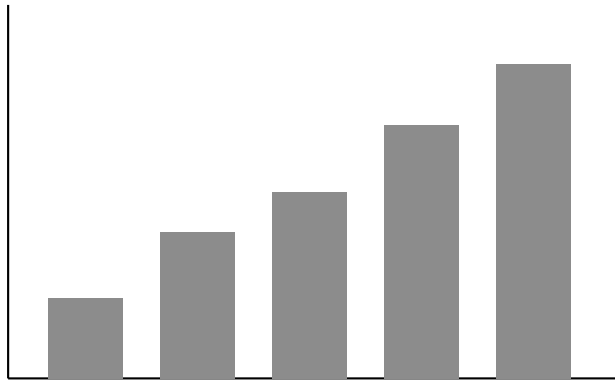


Figure 4. Rotation cost by tree size

In practice red-black trees perform fewer rotations than AVL trees on insertion, which is why most standard libraries use them.